

RETAIL DEMAND FORECASTING

STUDENT 2 — PROPER CONTRIBUTION DOCUMENTATION

Hive Data Modeling, Schema Design & SQL Analytics

Project	Retail Demand Forecasting
Student Role	Student 2 — Hive + Data Modeling
Input from Student 1	Raw retail_demand.csv stored in HDFS
Student 2 Environment	Docker-based Hadoop + Hive
Hive Database	retail_analytics
Hive Table	retail_demand
Records Verified	76,000
Columns	16

This document focuses specifically on Student 2's individual contribution and, in particular, explains how Student 2 extended the work completed by Student 1 into the Hive and SQL analytics layer.

Important integration clarification: Student 2 did not directly connect to Student 1's local HDFS machine. The supplied Student 2 execution log shows that the same project CSV was placed into Student 2's Docker/Hadoop HDFS, after which Hive was built on top of that HDFS copy. Therefore, the connection from Student 1 to Student 2 is a **logical/project pipeline connection**, not a live network connection between the two HDFS systems.

1. Student 2 Contribution — Overview

Student 1's responsibility was the **data ingestion and HDFS storage layer**. Student 2's responsibility begins after the raw dataset is available in HDFS: the data is exposed through Hive, modeled with a structured schema, verified, and analyzed using SQL.

Stage	Student 1	Student 2
Main role	Data ingestion + HDFS	Hive + Data Modeling + SQL Analytics
Input	Local CSV	Raw CSV in HDFS
Technology	Hadoop HDFS	Hive / Beeline
Main output	HDFS raw file	External Hive table + verified SQL results
Next hand-off	Student 2	Student 3 — Spark/PySpark

This makes Student 2 an extension of Student 1's storage layer: Student 1 makes the data available in distributed storage, while Student 2 makes that data structured and queryable.

2. How Student 2 Extended Student 1's Work

This is the most important part of the contribution. The extension happened in four logical steps:

- **1. Same project dataset:** Student 2 used the same **retail_demand.csv** dataset used for the project.
- **2. HDFS availability:** the dataset was placed in the Student 2 Docker Hadoop environment under **/data/retail/raw/**.
- **3. Hive mapping:** Student 2 created an external Hive table whose LOCATION points to **/data/retail/raw**.
- **4. SQL analytics:** Student 2 queried that Hive table and verified that all 76,000 records were available.

Thus the conceptual extension is:

```
Student 1:
CSV → HDFS raw storage
      |
      v
Student 2:
HDFS raw storage → Hive external table → SQL queries → verified analytics
      |
      v
Student 3:
Hive/HDFS data foundation → Spark/PySpark processing
```

The Student 2 execution log confirms the HDFS path was created and the CSV uploaded before the Hive table was created.

3. Important Integration Clarification

Because Student 1 and Student 2 may work on different systems, it is important not to describe the setup as if Student 2 directly accessed Student 1's local NameNode.

What happened	Correct description
Student 1's system	Local Hadoop/HDFS stored the project CSV.
Student 2's system	Docker-based Hadoop/Hive environment was used.
Physical HDFS connection	No direct connection between the two machines is shown in the Student 2 log.
Project-level connection	Student 2 used the same project dataset and reproduced it in the cloud/Docker HDFS environment.
Hive connection	Hive was directly connected to Student 2's HDFS path /data/retail/raw .

Therefore, in your final report you should say **"Student 2 extended Student 1's HDFS storage layer by creating the Hive schema and SQL analytics layer over the raw retail dataset"**, rather than saying **"Student 2 connected directly to Student 1's local HDFS"**.

4. Student 2 Environment Setup

Student 2 used a Docker-based Big Data environment. The supplied log shows all required services starting successfully with the full Docker Compose profile.

```
docker-compose --profile full up -d
```

Service	Status shown in log
hive-metastore	Running
resourcemanager	Running
jupyter	Running
pig	Running
spark-worker	Running
hive-metastore-postgres	Running
namenode	Running
hbase	Running
hive-server	Running
datanode	Running
spark-master	Running
nodemanager	Running

The environment used Apache Hive 2.3.2 with Beeline, Hadoop 2.7.4/YARN, and a PostgreSQL-backed Hive metastore.

5. Bringing the Dataset into Student 2's HDFS

Since Student 2's environment was separate from Student 1's local Hadoop system, the same project CSV had to be made available inside Student 2's HDFS environment.

First, the HDFS directory was created inside the NameNode container:

```
docker exec -it namenode hdfs dfs -mkdir -p /data/retail/raw
```

Then the CSV was copied from the Student 2 host machine into the NameNode container:

```
docker cp C:\bigdata-docker\retail_demand.csv namenode:/tmp/retail_demand.csv
```

The log reports a successful 6.35 MB transfer. The file was then uploaded from the container's temporary location into HDFS:

```
docker exec -it namenode hdfs dfs -put -f /tmp/retail_demand.csv /data/retail/raw/
```

Finally, the HDFS directory was checked:

```
docker exec -it namenode hdfs dfs -ls -h /data/retail/raw
```

HDFS file	Reported size
/data/retail/raw/retail_demand.csv	6.1 M

This step is the physical bridge between the shared project dataset and Student 2's Hive environment.

6. Connecting Hive to the HDFS Layer

After the CSV was available in HDFS, Student 2 entered the HiveServer container and connected to HiveServer2 using Beeline:

```
docker exec -it hive-server /bin/bash
beeline -u jdbc:hive2://localhost:10000
```

The log confirms a successful connection to **Apache Hive 2.3.2**. This is the point where Student 2's contribution moves from storage to the Hive query layer.

The connection architecture is:

HDFS

```

/data/retail/raw/retail_demand.csv
    |
    | Hive external table LOCATION
    v
Hive Metastore / HiveServer2
    |
    v
Beeline SQL interface

```

7. Creating the Hive Database

```

CREATE DATABASE IF NOT EXISTS retail_analytics;
USE retail_analytics;

```

The database **retail_analytics** provides a project-specific namespace for the Student 2 Hive layer.

8. Creating the External Hive Table

The key Student 2 contribution is the external Hive table. The table does not represent a second independent dataset; it provides a schema and SQL interface over the HDFS data.

```

CREATE EXTERNAL TABLE IF NOT EXISTS retail_demand (
  `date` STRING,
  store_id STRING,
  product_id STRING,
  category STRING,
  region STRING,
  inventory_level INT,
  units_sold INT,
  units_ordered INT,
  price DOUBLE,
  discount DOUBLE,
  weather_condition STRING,
  promotion STRING,
  competitor_pricing DOUBLE,
  seasonality STRING,
  epidemic STRING,
  demand INT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/data/retail/raw'
TBLPROPERTIES ('skip.header.line.count'='1');

```

The most important connection statement is **LOCATION '/data/retail/raw'**. This tells Hive where the HDFS data for the external table is located.

DDL component	Purpose
CREATE EXTERNAL TABLE	Creates Hive metadata without turning the raw file into a managed-table copy.
FIELDS TERMINATED BY ','	Matches the CSV delimiter.
STORED AS TEXTFILE	Reads the raw text/CSV storage format.
LOCATION '/data/retail/raw'	Connects Hive to the HDFS raw-data directory.
skip.header.line.count = 1	Prevents the CSV header from being counted as a data row.

9. Data Modeling — 16-Column Hive Schema

Column	Hive Type	Purpose
date	STRING	Date
store_id	STRING	Store identifier
product_id	STRING	Product identifier
category	STRING	Category
region	STRING	Region
inventory_level	INT	Inventory level

Column	Hive Type	Purpose
units_sold	INT	Units sold
units_ordered	INT	Units ordered
price	DOUBLE	Price
discount	DOUBLE	Discount
weather_condition	STRING	Weather condition
promotion	STRING	Promotion indicator
competitor_pricing	DOUBLE	Competitor price
seasonality	STRING	Seasonality
epidemic	STRING	Epidemic indicator
demand	INT	Target variable

The Hive schema preserves the complete 16-column project dataset and identifies **demand** as the target variable.

10. Verifying the Hive Table

Student 2 first verified that the table existed:

```
SHOW TABLES;
```

The result returned **retail_demand**. Student 2 then verified that actual rows could be read:

```
SELECT store_id, product_id, category, region, demand
FROM retail_demand
LIMIT 5;
```

store_id	product_id	category	region	demand
S001	P0001	Electronics	North	115
S001	P0002	Clothing	North	229
S001	P0003	Clothing	North	157
S001	P0004	Electronics	North	52
S001	P0005	Groceries	North	59

This proves that the Hive schema is correctly reading fields from the HDFS CSV.

11. Data Integrity Verification

The complete record count was then verified:

```
SELECT COUNT(*) FROM retail_demand;
```

Check	Result
Hive record count	76,000
Project dataset record count	76,000
Status	Verified

This is the main validation point connecting the Student 1 data-ingestion stage to the Student 2 Hive stage: the complete dataset is visible through the Hive table.

12. SQL Analytics 1 — Demand by Category

```
SELECT category,
       SUM(demand) AS total_demand,
       ROUND(AVG(demand), 2) AS avg_demand,
       ROUND(AVG(units_sold), 2) AS avg_units_sold
FROM retail_demand
GROUP BY category
ORDER BY total_demand DESC;
```

Category	Total Demand	Avg Demand	Avg Units Sold
Groceries	3,677,684	120.98	102.87
Clothing	1,369,456	112.62	94.64
Furniture	1,006,590	73.58	64.38
Toys	985,338	92.61	78.45
Electronics	889,036	97.48	83.04

This query supports the project objective by summarizing historical demand and sales at category level.

13. SQL Analytics 2 — Region and Seasonality

```
SELECT region,
       seasonality,
       SUM(demand) AS total_demand,
       ROUND(AVG(price), 2) AS avg_price,
       ROUND(AVG(discount), 2) AS avg_discount
FROM retail_demand
GROUP BY region, seasonality
ORDER BY region, total_demand DESC;
```

Region	Seasonality	Total Demand	Avg Price	Avg Discount
East	Winter	437,949	69.80	9.17
East	Summer	424,629	71.83	9.07
East	Autumn	387,126	70.10	9.13
East	Spring	368,620	68.68	8.96
North	Winter	856,949	63.11	9.17
North	Summer	830,530	65.04	9.06
North	Autumn	750,319	63.40	9.06
North	Spring	716,860	61.94	9.03
South	Winter	452,479	64.73	9.14
South	Summer	423,531	66.02	9.04
South	Autumn	380,251	63.90	9.13
South	Spring	369,210	62.90	9.09
West	Winter	424,153	77.78	9.11
West	Summer	397,849	78.25	9.08
West	Autumn	364,602	77.84	9.05
West	Spring	343,047	75.73	9.10

This query extends the demand analysis by combining geography and seasonality with price and discount measures.

14. SQL Analytics 3 — Weather and Promotion

```
SELECT weather_condition,
       promotion,
       COUNT(*) AS record_count,
       ROUND(AVG(demand), 2) AS avg_demand
FROM retail_demand
GROUP BY weather_condition, promotion
ORDER BY avg_demand DESC;
```

Weather	Promotion	Record Count	Avg Demand
Sunny	1	7,460	136.44
Cloudy	1	8,070	124.42
Rainy	1	5,726	112.21
Snowy	1	3,744	111.46

Weather	Promotion	Record Count	Avg Demand
Sunny	0	15,520	104.94
Cloudy	0	16,290	95.99
Rainy	0	11,774	86.79
Snowy	0	7,416	85.25

This query provides a descriptive comparison of demand under different weather and promotion combinations. The results should be described as observed patterns, not as proof of causation.

15. Execution Issues and How They Were Handled

Issue / Message	How it was handled
'hdfs' is not recognized in Windows CMD	HDFS commands were executed inside the NameNode container using docker exec.
Docker Compose 'version' attribute obsolete warning	Ignored because all 12 required containers started successfully.
SLF4J multiple bindings warning	Treated as a logging/classpath warning; Beeline connected successfully.
Hive-on-MR deprecated warning	Queries still executed successfully; it is a technology warning rather than a query failure.

Documenting these points is useful because it shows that the Student 2 workflow was executed and troubleshooting decisions were made rather than hiding the environment messages.

16. Complete Student 1 → Student 2 Connection

```

STUDENT 1
Local CSV
|
v
HDFS raw storage
/data/retail/raw/retail_demand.csv
|
| Same project dataset transferred/shared
v
STUDENT 2
Docker Hadoop/HDFS
/data/retail/raw/retail_demand.csv
|
| Hive external-table LOCATION
v
Hive database: retail_analytics
|
v
External table: retail_demand
|
+--> SELECT / LIMIT validation
|
+--> COUNT(*) = 76,000
|
+--> Category demand analysis
|
+--> Region + seasonality analysis
|
+--> Weather + promotion analysis
|
v
Verified Hive / SQL analytics layer
|
v
STUDENT 3
Spark / PySpark processing

```

This is the most accurate way to explain the team connection: Student 1 established the distributed storage stage; Student 2 recreated/accessed the same project dataset in the team's shared/cloud-style environment and built the Hive query layer on top of it.

17. Outputs Produced by Student 2

Output	Description
retail_analytics database	Project Hive database
retail_demand external table	Structured schema over HDFS raw data
16-column data model	Hive-compatible schema
76,000-row verification	Data integrity checkpoint
Category analysis	Demand, average demand, average units sold
Region-seasonality analysis	Demand, average price, average discount
Weather-promotion analysis	Record count and average demand
Student 3 hand-off	Verified Hive/HDFS data foundation

18. Formal Contribution Statement for the Final Report

You can use the following paragraph directly in your final project documentation:

Student 2 was responsible for the Hive Data Modeling and SQL Analytics layer of the Retail Demand Forecasting project. Building on the raw dataset ingested into HDFS during Student 1's stage, Student 2 made the same project dataset available in the Docker-based Hadoop environment and created an external Hive schema over the HDFS raw directory. A project-specific Hive database, `retail_analytics`, and external table, `retail_demand`, were created with all 16 dataset columns mapped to appropriate Hive data types. The table was validated using sample records and `SELECT COUNT(*)`, which returned 76,000 records. Student 2 then performed SQL analytics on category demand, regional seasonality with pricing and discounts, and weather-promotion demand patterns. This produced a verified Hive/SQL analytics layer that serves as the data foundation for Student 3's Spark/PySpark processing stage.

19. Viva Explanation — How Did You Extend Student 1?

Answer: Student 1 completed the data ingestion and HDFS storage part. Since my environment was separate, I used the same project CSV in my Docker Hadoop environment. I uploaded it into the HDFS raw directory `/data/retail/raw`. Then I connected Hive to that HDFS location using an external table with `LOCATION '/data/retail/raw'`. I created the `retail_analytics` database and `retail_demand` table, verified all 76,000 records, and performed SQL analytics. So my contribution extends the project from distributed storage to a structured Hive query and analytics layer.

If asked: Did you directly connect to Student 1's HDFS?

No. The two environments were separate. Student 1's local HDFS and Student 2's Docker HDFS are different HDFS instances. The integration was achieved by using the same project dataset and maintaining the same HDFS path/table schema logically. Hive was directly connected to Student 2's HDFS instance.

If asked: Why use an external table?

Because the CSV was already stored in HDFS. The external table provides Hive metadata and SQL access over that existing data without requiring the raw dataset to be treated as a separate managed-table copy.

20. Student 2 Completion Checklist

Task	Status
Docker Big Data services started	✓
HDFS raw directory created	✓
Same project CSV made available	✓
CSV uploaded to Student 2 HDFS	✓
HiveServer2 connection established	✓
Hive database created	✓

Task	Status
External table created	✓
All 16 columns modeled	✓
Sample records verified	✓
76,000 records verified	✓
Category SQL analysis	✓
Region/seasonality SQL analysis	✓
Weather/promotion SQL analysis	✓
Results documented	✓
Student 3 hand-off prepared	✓

21. Conclusion

Student 2 completed the second layer of the Retail Demand Forecasting pipeline. The work did not replace Student 1's HDFS contribution; it extended it by converting the raw distributed data into a structured Hive data model and queryable SQL layer.

The critical technical connection is the Hive external-table LOCATION pointing to **/data/retail/raw**. After this connection was established, Student 2 verified the complete 76,000-row dataset and executed three analytical workflows. The resulting Hive schema, verified data, and analytical outputs form the documented foundation for Student 3's Spark/PySpark stage.

For the final presentation, describe the integration as a **logical pipeline from HDFS → Hive → SQL analytics → Spark**. Do not claim that Student 2's Hive server directly accessed Student 1's local HDFS unless the team later creates an actual network connection.